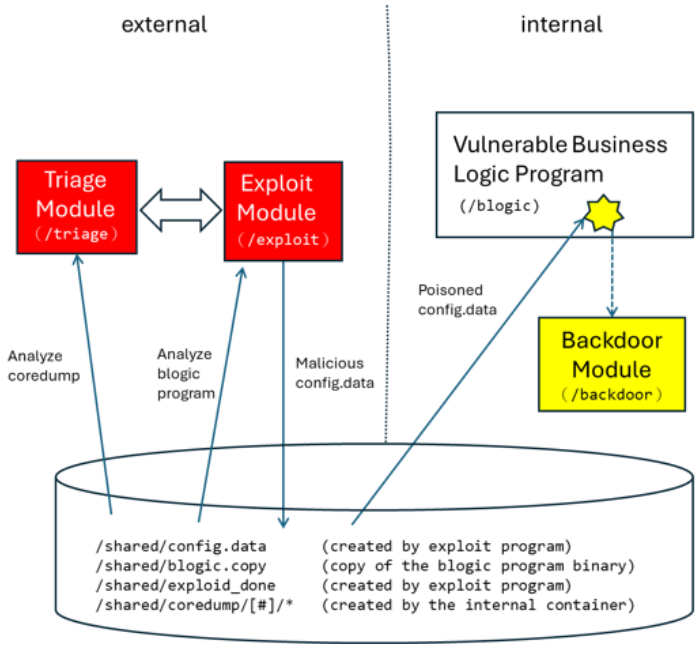
[2026 NS] Project - Automous APT Agent

In an advanced persistent threat (APT) scenario, it is crucial for the attacker to stay stealthy while gaining extensive access within the target organization, especially in settings where remote human control and command are limited or impossible.

For your term project, you will create an Autonomous APT Agent program. This agent will automatically detect and exploit software vulnerabilities, execute remote code to achieve persistence.

Project Overview

The environment for the APT Agent is assumed to include two main components: the exploit module and the backdoor module. The exploit module has the ability to modify the config file (/shared/config.data) of the business logic program. This capability allows the exploit module to exploit vulnerabilities within the business logic to facilitate the installation and activation of the backdoor module in the internal environment.



To simplify the development and the grading process. We have the following arrangements:

- OS and Software versions
  - Ubuntu Linux LTS 24.04.3 LTS x86_64
- There are two containers: the **external** container (**EC**) running the exploit module and the **internal** container (**IC**) running the business logic program.
- You will prepare the external container and submit it for grading.
  - Ensure that the exploit program can be executed from the path /exploit
  - Ensure that the triage program can be executed from the path /triage
- On the IC, the backdoor is preinstalled at /backdoor
- A shared volume (/shared) will be mounted on both the EC and the IC

File	Info
/shared/config.data	The blogic program in IC will read and process the data. The exploit in EC will try to put malformed data in config.data to exploit the vulnerability in the blogic program and trigger remote code execution of /backdoord on IC.
/shared/blogic.copy	A copy of the blogic program. This allows the exploit program to examine the vulnerability.
/shared/exploit_done	The exploit program will create the exploit_done file to indicate that it has finished tampering with config.data. Once IC detects the presence of exploit_done, it will launch the blogic program to process the config.data.
/shared/coredump/*	Coredump files from the crash of blogic in each grading round will be placed here.

- The backdoor module is preinstalled at /backdoor in the internal environment.

- The business logic program within the internal program will automatically restart after a crash.

Test and Grading Procedure

```
MaxTime = 30;  // 30 minutes
MaxRound = 60;

SetTimer(MaxTime, Terminate); //terminate test when MaxTime has been reached

Create a fresh shared volume /shared
Create a fresh external container IC
Run in EC

for round = 1..MaxRound
  (EC) /exploit begins execution
  (IC) Wait for the creation of /shared/exploit_done
  (IC) blogic reads and processes /shared/config.data (delete exploit_done
  (IC) If /backdoor on IC is successfully executed, mark the time and terminate the grading process
       Else put the coredump of blo
      gic under /shared/coredump/*
  (EC) /triage begins execution
```

The score depends on the total time your exploit program takes to penetrate the business logic program.

Project Phases & Milestones

Phase	Milestone	Objective
I	**Easy**	Stack-based buffer without boundary check Stack with executable permission non PIE executable ASLR disabled
II	**Medium**	Stack-based buffer without boundary check non PIE executable ASLR disabled

III	**Hard**	Stack-based buffer without boundary check non PIE executable ASLR enabled

Setup and Grader

lab.zip
30 Mar 2026, 02:07 AM

```
cd lab
docker build -t ic_image ./IC  #docker image
./docker.sh [1|2|3] # Create Docker containers for different project phases
```

Run grader.sh inside the EC you built.
The measurement method may be slightly modified
But make sure that /exploit and /triage are in the correct locations, and that /exploit modifies config.data and produces exploit_done.

For project related questions, please contact Mr. Cheng <armadillo.cs13@nycu.edu.tw>